

INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	TAREA GESTOR DE IMPRESIONES
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	3 - B
Alumnos participantes:	Altamirano Segovia Jullisa
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza

II. INFORME DE GUÍA PRÁCTICA

1. Actividades por desarrollar

Tarea Gestor de Impresiones

1. Ejercicio

Laboratorio: gestor de impresiones

Una cola conserva los documentos pendientes. Una pila registra los documentos ya impresos para permitir una recuperación.

Acción	Estructura	Operación sugerida	Resultado
Registrar documento	pendientes	offerLast(nombre)	Entra al final
Imprimir siguiente	pendientes	pollFirst()	Sale el más antiguo
Guardar impresión	historial	push(nombre)	Queda en la cima
Recuperar última	historial y pendientes	pop() y addFirst()	Vuelve al frente

Validar estructuras vacías antes de retirar.
Probar al menos seis operaciones combinadas.
Entregar código, salida y traza manual.

Use Deque<String> para ambas variables, pero respete el contrato de cada estructura.

2. Solución propuesta

Se define una única clase principal GestorImpresionesInteractivo que concentra la lógica del programa. Las dos estructuras se modelan con el mismo tipo concreto (ArrayDeque<String>) pero se usan con operaciones distintas: offerLast()/pollFirst() para el comportamiento de cola y push()/pop() para el comportamiento de pila. ArrayDeque se elige porque no permite elementos null y ofrece O(1) en inserción y extracción por ambos extremos. Antes de cada operación de retiro (pollFirst, pop) se valida que la estructura no esté vacía con isEmpty() Clases:

Clase	Responsabilidad
GestorImpresionesInteractivo	Clase principal: declara las estructuras pendientes e historial, ejecuta el menú de consola y coordina las operaciones de cada caso del switch.
Deque<String> pendientes	Cola FIFO de documentos por imprimir; se inserta con offerLast() y se extrae con pollFirst().
Deque<String> historial	Pila LIFO de documentos ya impresos; se inserta con push() y se extrae con pop().

Scanner	Lectura de la opción del menú y del nombre de cada documento.
---------	---

3. TDA

Se documentan los TDA/clases principales de cada ejercicio.

Valores:

- pendientes: Deque<String> — conjunto de documentos por imprimir en orden de llegada.
- historial: Deque<String> — pila de documentos ya impresos.
- opcion: int — opción seleccionada en el menú.
- doc, impreso, recuperado: String — nombres de documentos manipulados en cada operación.

Operaciones:

- registrarDocumento(nombre) : Gestor x String --> void
(pendientes.offerLast(nombre))
- imprimirSiguiente() : Gestor --> void (impreso = pendientes.pollFirst();
historial.push(impreso))
- recuperarUltima() : Gestor --> void (recuperado = historial.pop();
pendientes.addFirst(recuperado))
- mostrarEstado() : Gestor --> void (imprime pendientes e historial)
- estaVaciaPendientes() / estaVaciaHistorial() : Gestor --> boolean (isEmpty())

4. Relación entre requisitos y código

Requisito	Aplicación
Registrar documento (entra al final)	Caso 1 del switch: pendientes.offerLast(doc); el documento queda al final de la cola.
Imprimir siguiente (sale el más antiguo)	Caso 2 del switch: String impreso = pendientes.pollFirst(); respeta el orden FIFO.
Guardar impresión en el historial (queda en la cima)	Caso 2 del switch: historial.push(impreso); cada impresión queda en la cima de la pila.
Recuperar última (vuelve al frente)	Caso 3 del switch: pendientes.addFirst(historial.pop()); el documento regresa al frente de la cola.
Validar estructuras vacías antes de retirar	Condicionales if (!pendientes.isEmpty()) en el caso 2 y if (!historial.isEmpty()) en el caso 3.
Probar al menos seis operaciones combinadas	Traza manual de la prueba de escritorio (10 pasos combinando registro, impresión y recuperación).
Entregar código, salida y traza manual	Código completo en anexos, salida de consola y prueba de escritorio en la sección 6.10.

Usar Deque<String> para ambas variables respetando el contrato	Ambas se declaran como Deque<String> = new ArrayDeque<>(); la cola usa offerLast/pollFirst y la pila usa push/pop.
Mostrar estado de ambas estructuras	Caso 4 del switch: System.out.println de pendientes e historial.

5. Pasos de construcción

Paso 1

```
Scanner scanner = new Scanner(System.in);
Deque<String> pendientes = new ArrayDeque<>();
Deque<String> historial = new ArrayDeque<>();
```

Paso 2

```
case 1:
    System.out.print("Ingrese el nombre del documento: ");
    String doc = scanner.nextLine();
    pendientes.offerLast(doc);
    break;
```

Paso 3

```
case 2:
    if (!pendientes.isEmpty()) {
        String impreso = pendientes.pollFirst();
        historial.push(impreso);
        System.out.println("Imprimiendo: " + impreso);
    } else {
        System.out.println("No hay documentos pendientes.");
    }
    break;
```

Paso 4

```
case 3:
    if (!historial.isEmpty()) {
        String recuperado = historial.pop();
        pendientes.addFirst(recuperado);
        System.out.println("Recuperado al frente: " + recuperado);
    } else {
        System.out.println("No hay historial de impresiones.");
    }
    break;
```

Paso 5

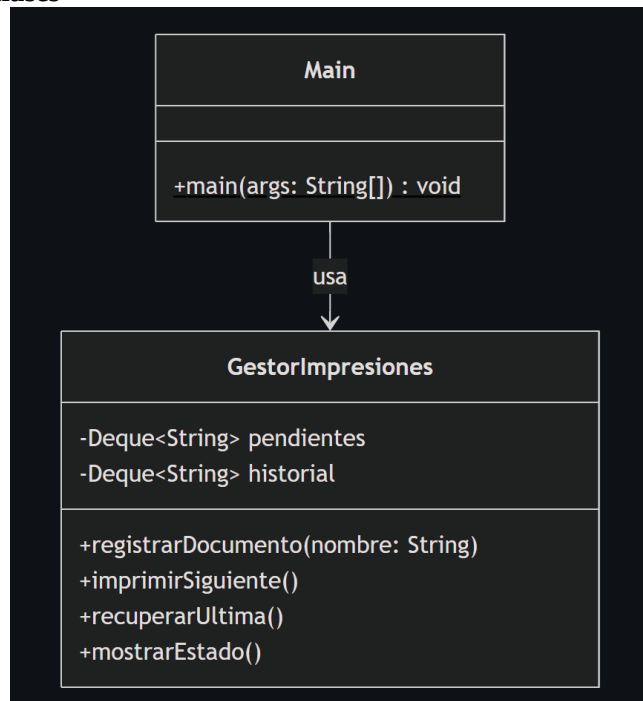
```
case 4:
    System.out.println("Pendientes: " + pendientes);
    System.out.println("Historial: " + historial);
```

6. Tipos de datos utilizados

Tipo	Categoría	Variables
String	Referencia	doc, impreso, recuperado; elementos de pendientes e historial

int	Primitivo	opcion
Deque<String>	Referencia (interfaz)	pendientes, historial
ArrayDeque<String>	Referencia (implementación)	instancia concreta asignada a ambas variables
Scanner	Referencia	scanner
boolean	Primitivo	resultado de isEmpty() en las validaciones

7. Diagrama de clases



8. Especificación del TDA

Estado: el gestor mantiene dos colecciones de cadenas. "pendientes" define el orden de impresión; "historial" define el orden inverso de impresión. No comparten estado con ninguna otra estructura.

Metodo	Resultado
registrarDocumento(nombre)	Inserta el documento al final de pendientes (offerLast); será impreso después de todos los que ya esperan.
imprimirSiguiete()	Si la cola no está vacía, retira el documento más antiguo (pollFirst) y lo apila en la cima del historial (push).
recuperarUltima()	Si la pila no está vacía, desapila la última impresión (pop) y la reinserta al frente de pendientes (addFirst).
mostrarEstado()	Imprime el contenido completo de pendientes e historial en su orden interno.

Invariantes:

- En pendientes, el documento que sale es siempre el de mayor antigüedad .
- En historial, el documento que sale es siempre el último que se imprimió .
- Ninguna estructura acepta elementos null .
- Un documento impreso nunca se pierde: siempre pasa primero por el historial antes de desaparecer del sistema.
- La recuperación solo es posible si el historial no está vacío; la impresión solo si pendientes no está vacía.

9. Pseudocódigo

```
INICIO
pendientes <- cola vacia
historial <- pila vacia
REPITE
    mostrar menu
    leer opcion
    SEGUN opcion HACER
        1: leer nombre
            pendientes.offerLast(nombre)
        2: SI pendientes NO vacia ENTONCES
            impreso <- pendientes.pollFirst()
            historial.push(impreso)
            imprimir "Imprimiendo: " + impreso
            SINO imprimir "No hay documentos pendientes."
        3: SI historial NO vacia ENTONCES
            recuperado <- historial.pop()
            pendientes.addFirst(recuperado)
            imprimir "Recuperado al frente: " + recuperado
            SINO imprimir "No hay historial de impresiones."
        4: imprimir pendientes e historial
        5: imprimir mensaje de salida
    FIN
```

10. Prueba de escritorio

Paso	Operación	Pendientes	Historial
1	Registrar informe.pdf	[informe.pdf]	[]
2	Registrar tarea.docx	[informe.pdf, tarea.docx]	[]
3	Registrar foto.png	[informe.pdf, tarea.docx, foto.png]	[]
4	Imprimir siguiente	[tarea.docx, foto.png]	[informe.pdf]
5	Imprimir siguiente	[foto.png]	[tarea.docx, informe.pdf]
6	Recuperar última	[tarea.docx, foto.png]	[informe.pdf]
7	Registrar resumen.txt	[tarea.docx, foto.png, resumen.txt]	[informe.pdf]
8	Imprimir siguiente	[foto.png, resumen.txt]	[tarea.docx, informe.pdf]

9	Imprimir siguiente	[resumen.txt]	[foto.png, tarea.docx, informe.pdf]
10	Recuperar última	[foto.png, resumen.txt]	[tarea.docx, informe.pdf]
11	Imprimir con cola vacía (validación)	[]	[resumen.txt]
12	Recuperar con historial lleno	[resumen.txt]	[]

2. Resultados obtenidos

El gestor de impresiones registró los documentos al final de la cola de pendientes y los imprimió siempre en orden de llegada, confirmando que `offerLast()` y `pollFirst()` sobre un `Deque` reproducen el comportamiento FIFO de una cola.

El historial de impresiones se comportó como pila: cada documento impreso quedó en la cima y la recuperación devolvió exactamente la última impresión, demostrando el comportamiento LIFO.

La operación de recuperación reinsertó el documento al frente de la cola, de modo que fue el siguiente en imprimirse.

Las validaciones de estructuras vacías funcionaron correctamente: intentar imprimir con la cola vacía o recuperar con la pila vacía muestra el mensaje correspondiente sin lanzar excepciones ni alterar el estado.

3. Anexos

Actualizaciones o mejoras del Software

Github:

<https://github.com/jullisaaltamirano2017-boop/TDA-Gestor-Impresora>